

Producenci i konsumenci z losową wielkością porcji

Szymon Iwaniuk

Teoria Współbieżności 2025/2026, laboratorium 4
30 października 2026

Spis treści

1 Wstęp **2**
1.1 Treść zadania . . . 2
1.2 Opis rozwiązania . . . 2

2 Specyfikacja narzędzi i sprzętu **2**
2.1 Metoda pomiaru . . . 3

3 Algorytmy **3**
3.1 Rozwiązanie naiwne . . . 4
3.2 Rozwiązanie z czterema warunkami . . . 5
3.3 Wyprzedzanie przy zwolnieniu blokady . . . 6

4 Implementacja **7**

5 Wyniki **16**
5.1 Czas wykonania a liczba wątków . . . 16
5.2 Różna liczba producentów i konsumentów . . . 17
5.3 Czas oczekiwania a wielkość porcji . . . 18
5.4 Podział pracy między wątki . . . 19

6 Wnioski **19**

1 Wstęp

1.1 Treść zadania

Producenci i konsumenci z losową ilością pobieranych i wstawianych porcji

- Bufor o rozmiarze $2M$
- Jest m producentów i n konsumentów
- Producent wstawia do bufora losową liczbę elementów (nie więcej niż M)
- Konsument pobiera losową liczbę elementów (nie więcej niż M)
- Zaimplementować przy pomocy monitorów Javy oraz mechanizmów Java Concurrency Utilities
- Przeprowadzić porównanie wydajności (np. czas wykonywania) vs. różne parametry, zrobić wykresy i je skomentować

1.2 Opis rozwiązania

Porównujemy cztery implementacje bufora:

- **monitor** – metody `synchronized` z `wait()` i `notifyAll()`,
- **ReentrantLock, 2 warunki** – osobne warunki `notFull` i `notEmpty`,
- **ReentrantLock, 4 warunki** – na miejsce w buforze czeka tylko pierwszy producent (konsument), a pozostali czekają za nim,
- **4 warunki + kolejka biletowa** – jak wyżej, ale kolejność wątków wyznacza numer pobrany przy wejściu.

Dwie pierwsze implementacje są rozwiązaniami „naiwnymi”: wątek, który chce dużej porcji, może być wielokrotnie wyprzedzany przez wątki chcące małych porcji. Rozwiązanie z czterema warunkami usuwa ten problem dla wielkości porcji, ale pomiary pokazały, że w Javie nie zapewnia kolejności między wątkami (rozdział 3.3). Dlatego dodaliśmy czwartą implementację z kolejką biletową i sprawdzamy, ile kosztuje pełna sprawiedliwość.

Razem ze sprawozdaniem zawarty jest kod w języku `Java` zawierający wszystkie implementacje, program wykonujący pomiary oraz skrypt służący do generowania zawartych w dokumencie wykresów.

2 Specyfikacja narzędzi i sprzętu

Program został napisany w języku `Java 27` i uruchomiony na maszynie wirtualnej `OpenJDK`, na komputerze z procesorem `Apple M4` (architektura `ARM64`, 10 rdzeni: 4 wydajnościowe i 6 energooszczędnych), `16 GB` zunifikowanej pamięci `LPDDR5` i systemem operacyjnym `macOS 26`. Do wygenerowania wykresów użyty został `Python` z biblioteką `matplotlib`.

2.1 Metoda pomiaru

Żeby każdy przebieg miał koniec, przez bufor przesyłamy ustaloną liczbę elementów $T = 20\,000 \cdot \frac{M+1}{2}$. Średnia porcja ma $\frac{M+1}{2}$ elementów, więc producenci wykonują łącznie około 20 000 wstawień niezależnie od M. Wątek losuje porcję, a bufor przycina ją do liczby elementów, które zostały do wstawienia (pobrania). Gdy nic nie zostało, `put` i `take` zwracają `false` i wątek kończy pracę. W pętlach wątków nie ma `Thread.sleep`, więc mierzymy koszt synchronizacji, a nie czas uśpienia.

Wszystkie wątki startują jednocześnie po zwolnieniu zatrzasku `CountDownLatch`. Czas mierzymy zegarem `System.nanoTime()` od startu do zakończenia ostatniego wątku (`join`). Przed pomiarami wykonujemy kilka przebiegów rozgrzewających kompilator JIT. Każdą konfigurację uruchamiamy trzy razy i bierzemy medianę. Dodatkowo każdy wątek mierzy czas każdego wywołania `put` i `take`, grupując wyniki według wielkości porcji w 10 przedziałach, i liczy swoje wywołania. Równomierność podziału pracy między wątki mierzymy indeksem Jaina

$$J(x_1, \ldots, x_k) = \frac{\left(\sum_{i=1}^{k} x_i\right)^2}{k \sum_{i=1}^{k} x_i^2},$$

gdzie x_i to liczba udanych wywołań i-tego producenta (konsumenta). $J = 1$, gdy wszystkie wątki wykonały tyle samo operacji, a $J = \frac{1}{k}$, gdy całą pracę wykonał jeden wątek. Na wykresach podajemy mniejszy z indeksów producentów i konsumentów.

Parametry: $M \in \{100, 1000, 10\,000\}$, $m, n \in \{1, 2, 4, 8, 16, 32\}$. Po każdym przebiegu sprawdzamy, że bufor jest pusty i przesłano dokładnie T elementów.

3 Algorytmy

W pseudokodzie s oznacza liczbę elementów w buforze, $2M - s$ liczbę wolnych miejsc, a r_p i r_k liczbę elementów, które zostały do wstawienia i do pobrania. Procedura TAKE jest symetryczna do PUT: zamienia rolami s i $2M - s$, r_p i r_k oraz warunki producentów i konsumentów, dlatego jej nie powtarzamy.

3.1 Rozwiązanie naiwne

Obie naiwne implementacje działają tak samo. Monitor ma jeden zbiór oczekujących, więc `notifyAll()` budzi wszystkie wątki. Wersja z `ReentrantLock` budzi tylko wątki drugiej strony, a własną stronę budzi jedynie po wyczerpaniu r_p, żeby pozostali producenci mogli się zakończyć.

Wstawianie w rozwiązaniu naiwnym

```
1:  Procedura PUT(k)
2:      Powtarzaj
3:          Jeżeli r_p = 0 wykonaj
4:              zwróć false
5:          k' ← min(k, r_p)
6:          Jeżeli 2M − s ≥ k' wykonaj
7:              wstaw k' elementów, r_p ← r_p − k'
8:              signalAll(notEmpty) ▷ monitor: notifyAll
9:              zwróć true
10:         await(notFull) ▷ monitor: wait
```

Producent chcący M elementów potrzebuje połowy bufora. Po każdym pobraniu budzą się wszyscy producenci, a pierwszy z nich, który dostanie blokadę, zajmuje zwolnione miejsce. Producenci z małymi porcjami mieszczą się częściej, więc producent z dużą porcją może czekać bardzo długo (zagłodzenie).

3.2 Rozwiązanie z czterema warunkami

Wątki jednej strony ustawiają się w kolejce na warunku *otherProducers*. Tylko pierwszy producent czeka na wolne miejsce na warunku *firstProducer*, więc nikt nie zajmie miejsca, na które on czeka. Po wyjściu oddaje miejsce następnemu w kolejce (wiersz 14).

Wstawianie w rozwiązaniu z czterema warunkami

```
1:  Procedura PUT(k)
2:      Dopóki producerFirst powtarzaj
3:          AWAIT(otherProducers)
4:      producerFirst ← true
5:      Powtarzaj
6:          Jeżeli r_p = 0 wykonaj
7:              wyjdź z pętli z wynikiem false
8:          k' ← min(k, r_p)
9:          Jeżeli 2M − s ≥ k' wykonaj
10:             wstaw k' elementów, r_p ← r_p − k'
11:             SIGNAL(firstConsumer)
12:             wyjdź z pętli z wynikiem true
13:         AWAIT(firstProducer)
14:     producerFirst ← false, SIGNAL(otherProducers)
15:     zwróć wynik
```

Konsument po pobraniu budzi *firstProducer*, czyli dokładnie jednego producenta. Gdy r_p spadnie do zera, każdy kolejny producent wychodzi od razu i budzi następnego, więc wszystkie wątki kończą pracę.

3.3 Wyprzedzanie przy zwolnieniu blokady

Domyślny `ReentrantLock` nie jest sprawiedliwy: wątek obudzony przez SIGNAL musi jeszcze ponownie zdobyć blokadę, a w tym czasie może ją przejąć dowolny inny wątek. Producent, który właśnie wyszedł z PUT w wierszu 14, od razu wywołuje PUT ponownie, widzi *producerFirst* = `false` i znów zostaje pierwszy, zanim obudzony wątek dostanie procesor. W naszym pomiarze dla $M = 1000$ i 16 producentów jeden wątek wykonał wszystkie 20 028 wstawień, a pozostałe 15 żadnego. Rozwiązanie jest więc sprawiedliwe względem wielkości porcji, ale głodzi wątki.

Kolejka biletowa usuwa ten problem. Każdy wątek przy wejściu pobiera kolejny numer i zostaje pierwszy dopiero wtedy, gdy obsługiwany jest jego numer. Kolejność obsługi jest więc kolejnością wejścia i nikt nie może się wepchnąć.

Wstawianie z kolejką biletową

```
1: Procedura PUT(k)
2:     b ← ticket,  ticket ← ticket + 1
3:     Dopóki b ≠ served powtarzaj
4:         AWAIT(otherProducers)
5:     wynik ← wiersze 5–13 rozwiązania z czterema warunkami
6:     served ← served + 1,  SIGNALALL(otherProducers)
7:     zwróć wynik
```

SIGNAL budzi dowolny czekający wątek, a nie koniecznie właściciela następnego numeru, dlatego w wierszu 6 budzimy wszystkich, a dalej przechodzi tylko jeden.

4 Implementacja

Poniżej znajduje się pełny kod programu. Każda klasa jest w osobnym pliku w katalogu `src`.

Klasa **Buffer** jest wspólną bazą wszystkich implementacji. Przechowuje elementy w buforze cyklicznym o rozmiarze $2M$ i liczniki elementów, które zostały do wstawienia i pobrania. Metoda `create` wybiera implementację po nazwie.

```
/**
 * Bounded buffer of capacity 2M with a fixed number of elements to pass through it.
 * put/take move min(want, remaining) elements and return false once nothing is left,
 * so every producer and consumer ends without being interrupted.
 */
abstract class Buffer {
    private final int[] data;
    private int head = 0;
    protected int size = 0;
    protected long toProduce;
    protected long toConsume;

    Buffer(int capacity, long total) {
        data = new int[capacity];
        toProduce = total;
        toConsume = total;
    }

    abstract boolean put(int want) throws InterruptedException;

    abstract boolean take(int want) throws InterruptedException;

    protected int free() {
        return data.length - size;
    }

    protected void insert(int n) {
        for (int i = 0; i < n; i++) {
            data[(head + size) % data.length] = 1;
            size++;
        }
        toProduce -= n;
    }

    protected void remove(int n) {
        for (int i = 0; i < n; i++) {
            data[head] = 0;
            head = (head + 1) % data.length;
            size--;
        }
        toConsume -= n;
    }

    static Buffer create(String impl, int capacity, long total) {
        return switch (impl) {
            case "monitor" -> new MonitorBuffer(capacity, total);
```

```
            case "lock" -> new LockBuffer(capacity, total);
            case "fair" -> new FairBuffer(capacity, total);
            case "ticket" -> new TicketBuffer(capacity, total);
            default -> throw new IllegalArgumentException(impl);
        };
    }
}
```

Kod 1: Klasa **Buffer** (`src/Buffer.java`)

Klasa **MonitorBuffer** to rozwiązanie naiwne na monitorze Javy. Cała metoda jest sekcją krytyczną, a `wait()` zwalnia monitor na czas oczekiwania.

```
/** Java monitor: synchronized methods, wait() and notifyAll() on a single wait set. */
class MonitorBuffer extends Buffer {
    MonitorBuffer(int capacity, long total) {
        super(capacity, total);
    }

    @Override
    synchronized boolean put(int want) throws InterruptedException {
        while (true) {
            if (toProduce == 0) return false;
            int n = (int) Math.min(want, toProduce);
            if (free() >= n) {
                insert(n);
                notifyAll();
                return true;
            }
            wait();
        }
    }

    @Override
    synchronized boolean take(int want) throws InterruptedException {
        while (true) {
            if (toConsume == 0) return false;
            int n = (int) Math.min(want, toConsume);
            if (size >= n) {
                remove(n);
                notifyAll();
                return true;
            }
            wait();
        }
    }
}
```

Kod 2: Klasa **MonitorBuffer** (`src/MonitorBuffer.java`)

Klasa **LockBuffer** to rozwiązanie naiwne z `ReentrantLock` i dwoma warunkami: producenci czekają na `notFull`, a konsumenci na `notEmpty`.

```
import java.util.concurrent.locks.Condition;
```

```java
import java.util.concurrent.locks.ReentrantLock;

/** Java Concurrency Utilities: ReentrantLock with separate conditions for producers and consumers. */
class LockBuffer extends Buffer {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    LockBuffer(int capacity, long total) {
        super(capacity, total);
    }

    @Override
    boolean put(int want) throws InterruptedException {
        lock.lock();
        try {
            while (true) {
                if (toProduce == 0) return false;
                int n = (int) Math.min(want, toProduce);
                if (free() >= n) {
                    insert(n);
                    notEmpty.signalAll();
                    if (toProduce == 0) notFull.signalAll(); // let the other producers finish
                    return true;
                }
                notFull.await();
            }
        } finally {
            lock.unlock();
        }
    }

    @Override
    boolean take(int want) throws InterruptedException {
        lock.lock();
        try {
            while (true) {
                if (toConsume == 0) return false;
                int n = (int) Math.min(want, toConsume);
                if (size >= n) {
                    remove(n);
                    notFull.signalAll();
                    if (toConsume == 0) notEmpty.signalAll(); // let the other consumers finish
                    return true;
                }
                notEmpty.await();
            }
        } finally {
            lock.unlock();
        }
    }
}
```

Kod 3: Klasa **LockBuffer** (`src/LockBuffer.java`)

Klasa **FairBuffer** to rozwiązanie z czterema warunkami. Blok `finally` zwalnia miejsce pierwszego producenta (konsumenta) niezależnie od tego, którą drogą metoda się kończy.

```
1  import java.util.concurrent.locks.Condition;
2  import java.util.concurrent.locks.ReentrantLock;
3
4  /**
5   * Four conditions: only the first producer (consumer) in line waits for space (elements),
6   * the rest queue behind it, so a large portion cannot be overtaken by smaller ones.
7   */
8  class FairBuffer extends Buffer {
9      private final ReentrantLock lock = new ReentrantLock();
10     private final Condition firstProducer = lock.newCondition();
11     private final Condition otherProducers = lock.newCondition();
12     private final Condition firstConsumer = lock.newCondition();
13     private final Condition otherConsumers = lock.newCondition();
14     private boolean producerFirst = false;
15     private boolean consumerFirst = false;
16
17     FairBuffer(int capacity, long total) {
18         super(capacity, total);
19     }
20
21     @Override
22     boolean put(int want) throws InterruptedException {
23         lock.lock();
24         try {
25             while (producerFirst) otherProducers.await();
26             producerFirst = true;
27             try {
28                 while (true) {
29                     if (toProduce == 0) return false;
30                     int n = (int) Math.min(want, toProduce);
31                     if (free() >= n) {
32                         insert(n);
33                         firstConsumer.signal();
34                         return true;
35                     }
36                     firstProducer.await();
37                 }
38             } finally {
39                 producerFirst = false;
40                 otherProducers.signal();
41             }
42         } finally {
43             lock.unlock();
44         }
45     }
46
47     @Override
```

```
48      boolean take(int want) throws InterruptedException {
49          lock.lock();
50          try {
51              while (consumerFirst) otherConsumers.await();
52              consumerFirst = true;
53              try {
54                  while (true) {
55                      if (toConsume == 0) return false;
56                      int n = (int) Math.min(want, toConsume);
57                      if (size >= n) {
58                          remove(n);
59                          firstProducer.signal();
60                          return true;
61                      }
62                      firstConsumer.await();
63                  }
64              } finally {
65                  consumerFirst = false;
66                  otherConsumers.signal();
67              }
68          } finally {
69              lock.unlock();
70          }
71      }
72  }
```

Kod 4: Klasa **FairBuffer** (`src/FairBuffer.java`)

Klasa **TicketBuffer** różni się od **FairBuffer** tylko wejściem i wyjściem: wątek czeka na swój numer, a przy wyjściu obsługiwany jest następny numer.

```
1   import java.util.concurrent.locks.Condition;
2   import java.util.concurrent.locks.ReentrantLock;
3
4   /**
5    * FairBuffer with a ticket queue: each thread takes a number and becomes first only when
6    * its number is served, so a thread that grabs the lock right after leaving cannot jump
7    * ahead of the threads already waiting.
8    */
9   class TicketBuffer extends Buffer {
10      private final ReentrantLock lock = new ReentrantLock();
11      private final Condition firstProducer = lock.newCondition();
12      private final Condition otherProducers = lock.newCondition();
13      private final Condition firstConsumer = lock.newCondition();
14      private final Condition otherConsumers = lock.newCondition();
15      private long producerTicket = 0, producerServed = 0;
16      private long consumerTicket = 0, consumerServed = 0;
17
18      TicketBuffer(int capacity, long total) {
19          super(capacity, total);
20      }
21
22      @Override
23      boolean put(int want) throws InterruptedException {
```

```
24          lock.lock();
25          try {
26              long ticket = producerTicket++;
27              while (ticket != producerServed) otherProducers.await();
28              try {
29                  while (true) {
30                      if (toProduce == 0) return false;
31                      int n = (int) Math.min(want, toProduce);
32                      if (free() >= n) {
33                          insert(n);
34                          firstConsumer.signal();
35                          return true;
36                      }
37                      firstProducer.await();
38                  }
39              } finally {
40                  producerServed++;
41                  otherProducers.signalAll(); // only the owner of the next ticket proceeds
42              }
43          } finally {
44              lock.unlock();
45          }
46      }
47
48      @Override
49      boolean take(int want) throws InterruptedException {
50          lock.lock();
51          try {
52              long ticket = consumerTicket++;
53              while (ticket != consumerServed) otherConsumers.await();
54              try {
55                  while (true) {
56                      if (toConsume == 0) return false;
57                      int n = (int) Math.min(want, toConsume);
58                      if (size >= n) {
59                          remove(n);
60                          firstProducer.signal();
61                          return true;
62                      }
63                      firstConsumer.await();
64                  }
65              } finally {
66                  consumerServed++;
67                  otherConsumers.signalAll();
68              }
69          } finally {
70              lock.unlock();
71          }
72      }
73  }
```

Kod 5: Klasa **TicketBuffer** (`src/TicketBuffer.java`)

Klasa **Main** tworzy wątki producentów i konsumentów dla wszystkich konfiguracji, mierzy czas, liczy indeks Jaina i zapisuje wyniki do plików CSV, z których skrypt w Pythonie rysuje wykresy.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.SplittableRandom;
import java.util.concurrent.CountDownLatch;

/**
 * Runs every implementation for a grid of M, producer and consumer counts and writes
 * csv/times.csv (wall time) and csv/waits.csv (time spent in put/take per portion size).
 */
public class Main {
    static final String[] IMPLS = {"monitor", "lock", "fair", "ticket"};
    static final int[] PORTIONS = {100, 1000, 10000};
    static final int[] THREADS = {1, 2, 4, 8, 16, 32};
    static final int OPS = 20_000;   // expected number of put calls in one run
    static final int REPS = 3;
    static final int BUCKETS = 10;   // portion sizes 1..M split into equal ranges

    /** Wait time histogram of one thread, merged after the run. */
    static final class Stats {
        final long[] count = new long[BUCKETS];
        final long[] sumNs = new long[BUCKETS];
        final long[] maxNs = new long[BUCKETS];

        void record(int bucket, long ns) {
            count[bucket]++;
            sumNs[bucket] += ns;
            maxNs[bucket] = Math.max(maxNs[bucket], ns);
        }

        void merge(Stats o) {
            for (int b = 0; b < BUCKETS; b++) {
                count[b] += o.count[b];
                sumNs[b] += o.sumNs[b];
                maxNs[b] = Math.max(maxNs[b], o.maxNs[b]);
            }
        }
    }

    /** Jain's index of calls per thread: 1 when all threads did the same work, 1/k when one did all. */
    static double jain(Stats[] stats, int from, int to) {
        double sum = 0, sumSq = 0;
        for (int t = from; t < to; t++) {
            long calls = Arrays.stream(stats[t].count).sum();
            sum += calls;
            sumSq += (double) calls * calls;
        }
```

```
51          return sumSq == 0 ? 1 : sum * sum / ((to - from) * sumSq);
52      }
53
54      record Result(long timeNs, Stats producers, Stats consumers, double jainProducers, double
        ↪ jainConsumers) {}
55
56      static Result run(String impl, int m, int producers, int consumers, long seed) throws
        ↪ InterruptedException {
57          long total = (long) OPS * (m + 1) / 2;
58          Buffer buffer = Buffer.create(impl, 2 * m, total);
59          CountDownLatch ready = new CountDownLatch(producers + consumers);
60          CountDownLatch start = new CountDownLatch(1);
61          List<Thread> threads = new ArrayList<>();
62          Stats[] stats = new Stats[producers + consumers];
63
64          for (int t = 0; t < producers + consumers; t++) {
65              boolean producer = t < producers;
66              Stats s = stats[t] = new Stats();
67              SplittableRandom rnd = new SplittableRandom(seed * 1_000 + t);
68              threads.add(new Thread(() -> {
69                  try {
70                      ready.countDown();
71                      start.await();
72                      while (true) {
73                          int want = rnd.nextInt(m) + 1;
74                          long t0 = System.nanoTime();
75                          boolean ok = producer ? buffer.put(want) : buffer.take(want);
76                          if (!ok) break;
77                          s.record((want - 1) * BUCKETS / m, System.nanoTime() - t0);
78                      }
79                  } catch (InterruptedException e) {
80                      Thread.currentThread().interrupt();
81                  }
82              }));
83          }
84          for (Thread th : threads) th.start();
85          ready.await();
86          long t0 = System.nanoTime();
87          start.countDown();
88          for (Thread th : threads) th.join();
89          long elapsed = System.nanoTime() - t0;
90
91          if (buffer.size != 0 || buffer.toProduce != 0 || buffer.toConsume != 0)
92              throw new IllegalStateException(impl + ": buffer not drained");
93          Stats p = new Stats(), c = new Stats();
94          for (int t = 0; t < stats.length; t++) (t < producers ? p : c).merge(stats[t]);
95          return new Result(elapsed, p, c, jain(stats, 0, producers), jain(stats, producers, stats.length)
        ↪ );
96      }
97
98      static void writeWaits(PrintWriter w, String impl, int m, int producers, int consumers, String role,
        ↪  Stats s) {
99          for (int b = 0; b < BUCKETS; b++) {
```

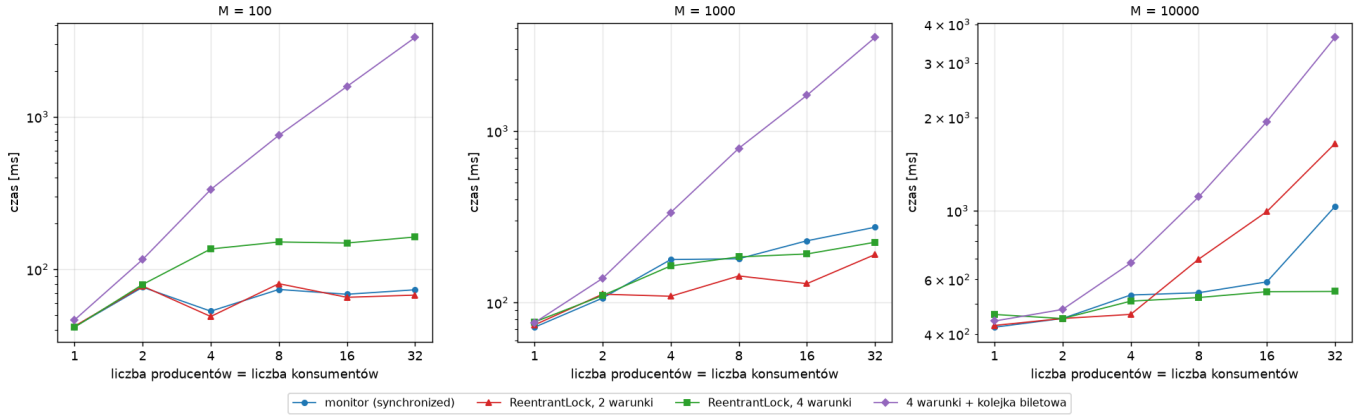
```
100             if (s.count[b] == 0) continue;
101             w.printf("%s,%d,%d,%d,%s,%d,%d,%d,%.3f,%.3f%n", impl, m, producers, consumers, role,
102                     b * m / BUCKETS + 1, (b + 1) * m / BUCKETS, s.count[b],
103                     s.sumNs[b] / 1e3 / s.count[b], s.maxNs[b] / 1e3);
104         }
105     }
106
107     public static void main(String[] args) throws IOException, InterruptedException {
108         String out = args.length > 0 ? args[0] : "csv";
109
110         // JIT warm-up, results discarded
111         for (String impl : IMPLS)
112             for (int i = 0; i < 5; i++) run(impl, 1000, 4, 4, i);
113
114         try (PrintWriter times = new PrintWriter(new FileWriter(out + "/times.csv"));
115              PrintWriter waits = new PrintWriter(new FileWriter(out + "/waits.csv"))) {
116             times.println("impl,M,producers,consumers,rep,time_ms,jain_producers,jain_consumers");
117             waits.println("impl,M,producers,consumers,role,portion_lo,portion_hi,count,mean_us,max_us");
118             for (int m : PORTIONS)
119                 for (int p : THREADS)
120                     for (int c : THREADS) {
121                         for (String impl : IMPLS) {
122                             Stats ps = new Stats(), cs = new Stats();
123                             double[] ms = new double[REPS];
124                             for (int r = 0; r < REPS; r++) {
125                                 Result res = run(impl, m, p, c, r);
126                                 ms[r] = res.timeNs() / 1e6;
127                                 times.printf("%s,%d,%d,%d,%d,%.3f,%.4f,%.4f%n", impl, m, p, c, r, ms[r],
    ↪ res.jainProducers(), res.jainConsumers());
128                                 ps.merge(res.producers());
129                                 cs.merge(res.consumers());
130                             }
131                             writeWaits(waits, impl, m, p, c, "producer", ps);
132                             writeWaits(waits, impl, m, p, c, "consumer", cs);
133                             Arrays.sort(ms);
134                             System.out.printf("M=%-5d p=%-2d c=%-2d %-7s median %8.1f ms%n", m, p, c,
    ↪ impl, ms[REPS / 2]);
135                         }
136                         times.flush();
137                     }
138         }
139     }
140 }
```

Kod 6: Klasa **Main** (`src/Main.java`)

5 Wyniki

5.1 Czas wykonania a liczba wątków

Rysunek 1 pokazuje czas przesłania wszystkich elementów, gdy producentów i konsumentów jest tyle samo. Oś czasu jest logarytmiczna, bo kolejka biletowa jest wolniejsza o rząd wielkości.



Rysunek 1: Czas wykonania w zależności od liczby wątków dla $m = n$

Dla jednego producenta i jednego konsumenta wszystkie implementacje mają ten sam czas, bo nie ma rywalizacji o blokadę. Czas rośnie z M (42 ms dla $M = 100$, 420 ms dla $M = 10\,000$), bo przy stałej liczbie wywołań każde wywołanie kopiuje średnio $\frac{M+1}{2}$ elementów.

Rozwiązania naiwne zwalniają umiarkowanie dla małych porcji, ale dla $M = 10\,000$ i 32 parach wątków są 2–4 razy wolniejsze niż dla jednej pary. Po każdej operacji `notifyAll()` lub `signalAll()` budzi wszystkie wątki drugiej strony, choć zwykle tylko jeden z nich może cokolwiek zrobić. Rozwiązanie z czterema warunkami budzi dokładnie jeden wątek i jego czas prawie nie zależy od liczby wątków. Kolejka biletowa jest najwolniejsza, a jej czas rośnie liniowo z liczbą wątków.

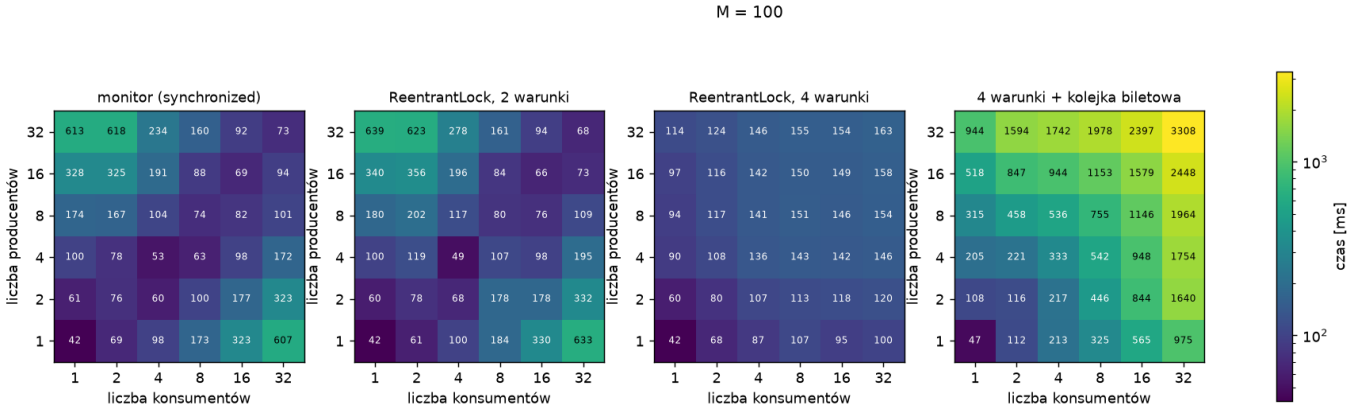
Tabela 1 zestawia czasy i indeksy Jaina dla 16 producentów i 16 konsumentów.

Implementacja	$M = 100$		$M = 1000$		$M = 10\,000$	
	czas [ms]	J	czas [ms]	J	czas [ms]	J
monitor (`synchronized`)	68,6	0,66	229,3	0,22	589,4	0,23
`ReentrantLock`, 2 warunki	**65,5**	0,65	**129,0**	0,47	994,2	0,13
`ReentrantLock`, 4 warunki	148,7	0,06	192,1	0,06	**547,9**	0,06
4 warunki + kolejka biletowa	1578,7	**1,00**	1615,8	**1,00**	1939,6	**1,00**

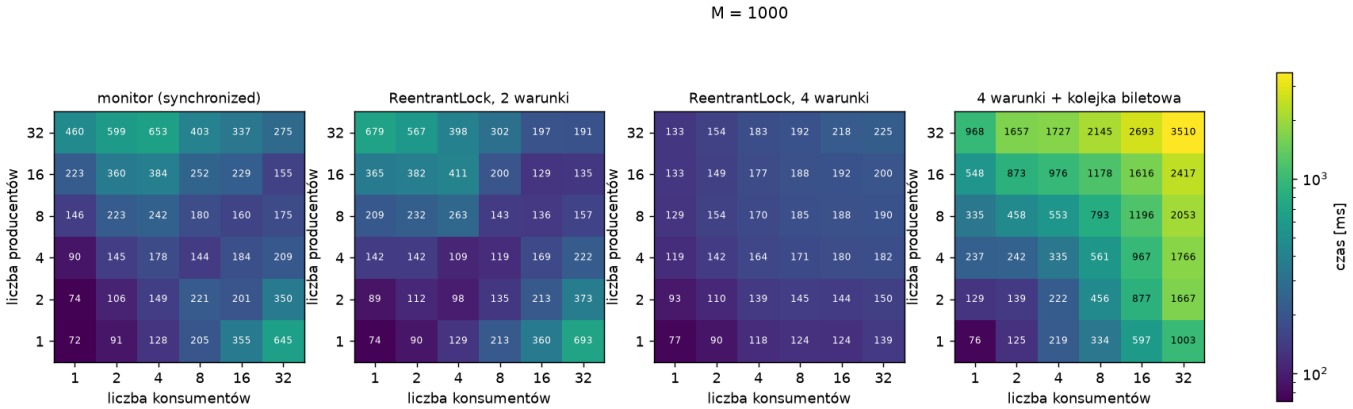
Tabela 1: Czas wykonania i indeks Jaina J dla 16 producentów i 16 konsumentów

5.2 Różna liczba producentów i konsumentów

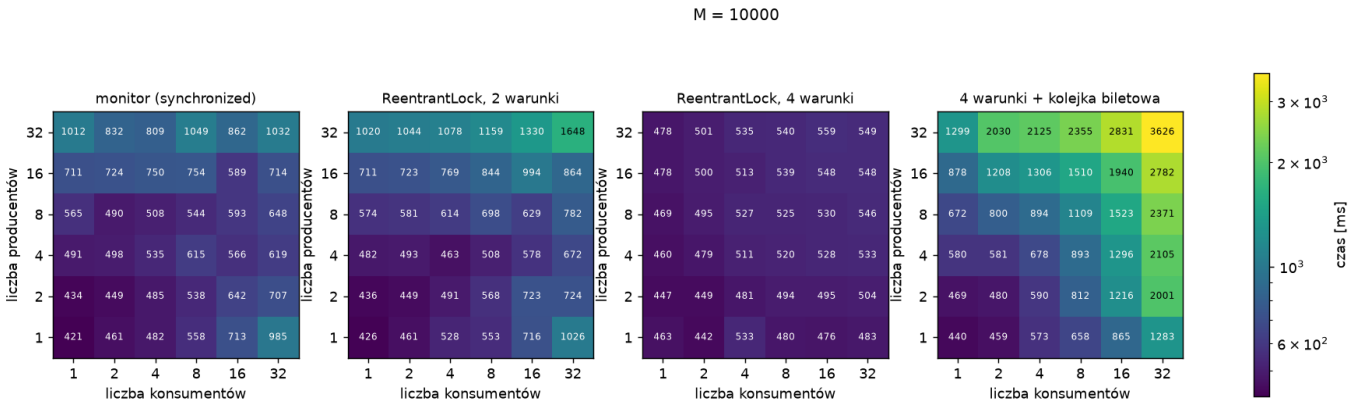
Rysunki 2–4 pokazują mediany czasu dla wszystkich par (m, n). Wszystkie panele jednego rysunku mają wspólną, logarytmiczną skalę kolorów.



Rysunek 2: Czas wykonania [ms] w zależności od liczby producentów i konsumentów, $M = 100$



Rysunek 3: Czas wykonania [ms] w zależności od liczby producentów i konsumentów, $M = 1000$

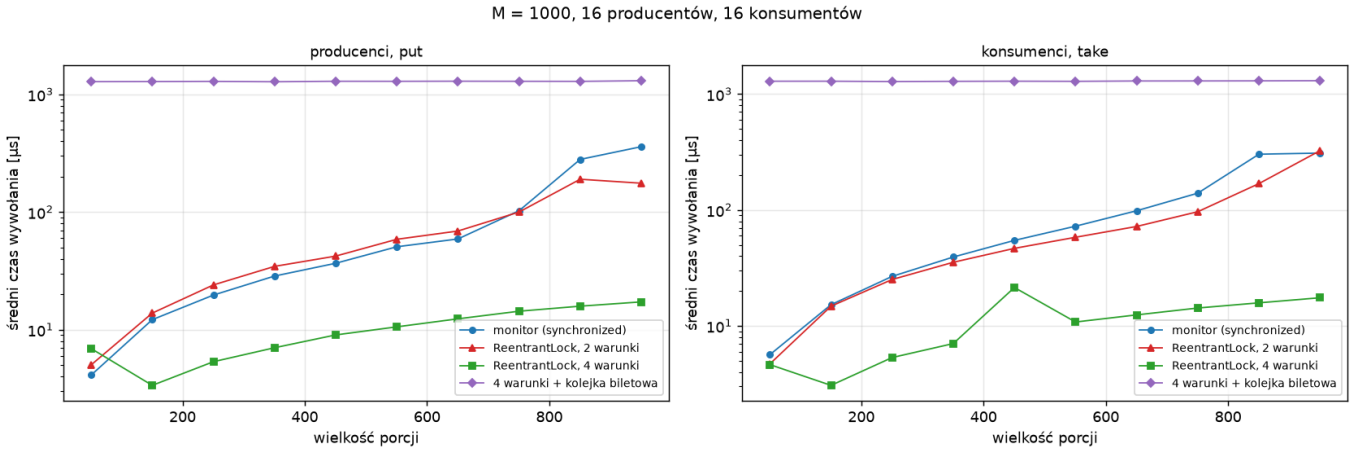


Rysunek 4: Czas wykonania [ms] w zależności od liczby producentów i konsumentów, $M = 10\,000$

Rozwiązania naiwne mają najgorsze czasy w rogach, czyli przy jednym producencie i 32 konsumentach albo odwrotnie: dla $M = 1000$ jest to 460–690 ms, a przy 32 parach wątków tylko 190–275 ms. Jedyny producent po każdym wstawieniu budzi 32 konsumentów, z których większość nie znajduje wystarczającej liczby elementów i od razu znów czeka. W rozwiązaniu z czterema warunkami budzi się tylko pierwszy konsument, dlatego te same konfiguracje zajmują 130–140 ms. W kolejce biletowej czas rośnie z każdą z liczb wątków, bo dłuższa kolejka to więcej przełączeń wątków na każdą operację.

5.3 Czas oczekiwania a wielkość porcji

Rysunek 5 pokazuje średni czas jednego wywołania `put` i `take` w zależności od wielkości porcji dla $M = 1000$ i 16 par wątków.

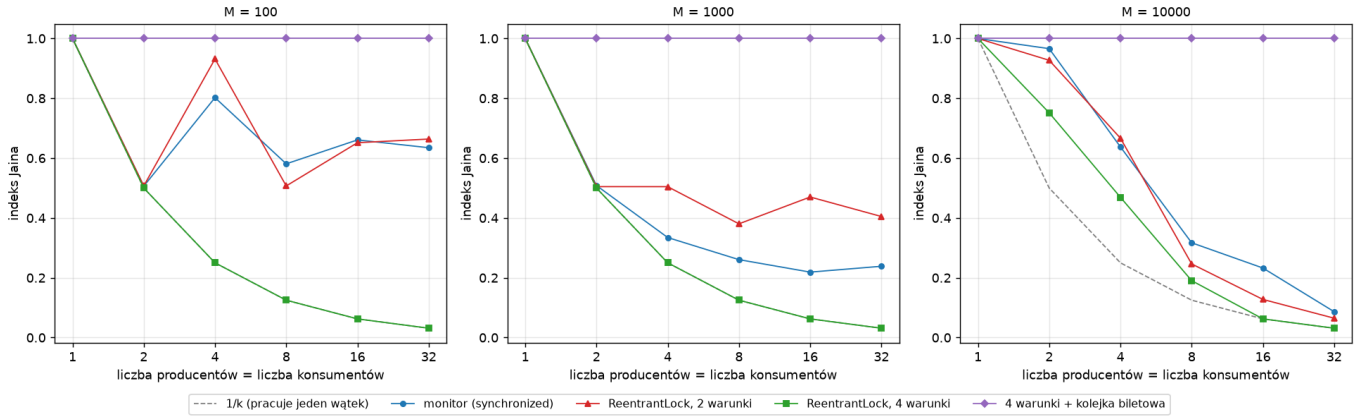


Rysunek 5: Średni czas wywołania w zależności od wielkości porcji

W rozwiązaniach naiwnych wywołanie z porcją bliską M trwa średnio 175–360 µs, a z porcją do 100 elementów 4–6 µs, czyli 40–80 razy krócej. Wątki z dużymi porcjami są wyprzedzane przez wątki z małymi, zgodnie z opisem z rozdziału 3.1. W rozwiązaniu z czterema warunkami czas rośnie tylko z 7 do 17 µs, mniej więcej proporcjonalnie do liczby kopiowanych elementów. W kolejce biletowej każde wywołanie trwa około 1,3 ms niezależnie od porcji, bo wątek czeka, aż obsłużone zostaną wszystkie wątki przed nim w kolejce.

5.4 Podział pracy między wątki

Rysunek 6 pokazuje indeks Jaina. Przerywana linia $\frac{1}{k}$ odpowiada sytuacji, w której całą pracę wykonuje jeden wątek.



Rysunek 6: Indeks Jaina w zależności od liczby wątków dla $m = n$

Dla $M \leq 1000$ rozwiązanie z czterema warunkami leży dokładnie na linii $\frac{1}{k}$: z powodu opisanego w rozdziale 3.3 jeden producent i jeden konsument wykonują wszystkie operacje, a pozostałe wątki przez cały przebieg czekają. Stąd też jego stały czas na rysunku 1: niezależnie od k pracuje jedna para wątków. Dla $M = 10\,000$ kopiowanie porcji trwa na tyle długo, że obudzony wątek czasem zdąży zdobyć blokadę, ale od 8 par wątków indeks i tak jest bliski $\frac{1}{k}$. Rozwiązania naiwne dzielą pracę lepiej, ale też nierówno ($J = 0{,}06$–$0{,}7$ dla 8 i więcej par wątków). Tylko kolejka biletowa daje $J = 1$ we wszystkich konfiguracjach. Monitor `synchronized` również nie gwarantuje kolejności wejścia, więc i on pozwala wątkowi, który właśnie wyszedł, od razu wrócić.

6 Wnioski

Monitor Javy i `ReentrantLock` z dwoma warunkami działają podobnie. Dla małych porcji są najszybsze, ale wątki z dużymi porcjami czekają kilkadziesiąt razy dłużej niż wątki z małymi, a przy nierównej liczbie producentów i konsumentów budzenie wszystkich czekających wątków sprawia, że działają 4–5 razy dłużej niż rozwiązanie z czterema warunkami. Osobne warunki dla producentów i konsumentów pomagają dla $M \leq 1000$, ale dla $M = 10\,000$ i wielu wątków wersja z `ReentrantLock` okazała się wolniejsza od monitora.

Rozwiązanie z czterema warunkami usuwa zależność czasu oczekiwania od wielkości porcji i najlepiej skaluje się dla dużych porcji. Nie jest jednak sprawiedliwe względem wątków: domyślny `ReentrantLock` pozwala wątkowi, który właśnie zwolnił blokadę, natychmiast ją odzyskać, więc praktycznie całą pracę wykonuje jeden producent i jeden konsument. Jego dobra wydajność wynika głównie z tego, że pozostałe wątki prawie nie biorą udziału w pracy.

Kolejka biletowa jest jedyną implementacją, w której każdy wątek wykonuje tyle samo operacji i czeka tyle samo niezależnie od porcji. Kosztuje to jednak czas: dla 32 par wątków jest 2–7 razy wolniejsza dla $M = 10\,000$ i do 50 razy wolniejsza dla $M = 100$, bo każda operacja wymaga przekazania pierwszego miejsca innemu, uśpionemu wątkowi. Wybór implementacji zależy więc od tego, czy ważniejsza jest przepustowość, czy gwarancja, że żaden wątek nie zostanie zagłodzony.

Literatura

[1] Per Brinch Hansen: *Java's Insecure Parallelism*. ACM SIGPLAN Notices, kwiecień 1999.

[2] `https://docs.oracle.com/en/java/javase/27/docs/api/java.base/java/util/concurrent/locks/Condition.html`

[3] `https://jenkov.com/tutorials/java-util-concurrent/index.html`